

MLFlow

Introduction à MLflow

Qu'est-ce que MLflow ?

Définition

MLflow est une plateforme open-source pour gérer le cycle de vie complet du Machine Learning.

Composant	Fonction	Exemple d'usage
Tracking	Enregistrer expériences	Logger accuracy, loss
Projects	Packager le code	Reproduire une expérience
Models	Format standard	Déployer n'importe où
Registry	Versionner modèles	Staging → Production

Problèmes résolus par MLflow

Sans MLflow :

- "Quels paramètres ont donné 95% d'accuracy ?"
- "Où est le code de l'expérience du 15 mars ?"
- "Comment reproduire ce modèle ?"
- "Quelle version est en production ?"

Problèmes résolus par MLflow

Avec MLflow :

- Tous les paramètres logged automatiquement
- Code versionné avec Git + MLflow
- Reproduction en 1 commande
- Registry avec stages clairs

Installation

Installation basique

```
# Via pip
pip install mlflow

# Vérifier l'installation
mlflow --version

# Version recommandée : 2.8+
```

Installation complète

```
# Avec toutes les dépendances
pip install mlflow[extras]

# Ou sélectivement
pip install mlflow scikit-learn pandas matplotlib

# Pour PostgreSQL backend
pip install mlflow psycopg2-binary

# Pour AWS
pip install mlflow boto3
```

Lancer l'interface MLflow UI

```
# Lancer avec stockage local
mlflow ui

# Avec port personnalisé
mlflow ui --port 5001

# Avec backend spécifique
mlflow ui --backend-store-uri sqlite:///mlflow.db

# Interface accessible sur http://localhost:5000
```

Concepts fondamentaux

Run

Un **run** = une exécution d'un script ML.

```
import mlflow

# Démarrer un run
with mlflow.start_run():
    # Votre code ML ici
    pass
```

Experiment

Un **experiment** = groupe de runs liés.

```
# Créer/utiliser un experiment
mlflow.set_experiment("my-classification-project")

with mlflow.start_run():
    # Ce run sera dans "my-classification-project"
    pass
```

Parameters

Parameters = hyperparamètres du modèle.

```
with mlflow.start_run():  
    mlflow.log_param("learning_rate", 0.01)  
    mlflow.log_param("n_estimators", 100)  
    mlflow.log_param("max_depth", 5)
```

Metrics

Metrics = résultats de l'entraînement.

```
with mlflow.start_run():  
    mlflow.log_metric("accuracy", 0.95)  
    mlflow.log_metric("f1_score", 0.93)  
    mlflow.log_metric("precision", 0.96)
```

Artifacts

Artifacts = fichiers générés (modèles, graphiques, data).

```
with mlflow.start_run():  
    # Logger un fichier  
    mlflow.log_artifact("confusion_matrix.png")  
  
    # Logger un dossier  
    mlflow.log_artifacts("plots/")  
  
    # Logger un modèle  
    mlflow.sklearn.log_model(model, "model")
```

Premier script MLflow

Explorer l'interface MLflow UI

Page des Experiments

Fonctionnalités :

- Tableau de tous les runs
- Recherche et filtres
- Comparaison visuelle
- Export CSV

Détails d'un run

Cliquer sur un run pour voir :

- **Parameters** : Tous les hyperparamètres
- **Metrics** : Accuracy, loss, etc.
- **Artifacts** : Modèles, graphiques
- **Tags** : Métadonnées custom
- **Source** : Code source et Git commit

Comparer des runs

1. Sélectionner plusieurs runs (checkbox)
2. Cliquer sur "Compare"
3. Vue côte à côte des métriques
4. Graphiques de comparaison

Logging avancé

Logger plusieurs métriques à la fois

```
with mlflow.start_run():  
    mlflow.log_metrics({  
        "accuracy": 0.95,  
        "precision": 0.93,  
        "recall": 0.94,  
        "f1_score": 0.935  
    })
```

Logger des métriques par epoch

```
with mlflow.start_run():  
    for epoch in range(10):  
        loss = train_one_epoch()  
        mlflow.log_metric("loss", loss, step=epoch)
```

Logger des paramètres en batch

```
params = {  
    "learning_rate": 0.01,  
    "batch_size": 32,  
    "epochs": 10,  
    "optimizer": "adam"  
}  
  
with mlflow.start_run():  
    mlflow.log_params(params)
```

Logger des graphiques

```
import matplotlib.pyplot as plt

with mlfow.start_run():
    # Créer un graphique
    plt.figure(figsize=(8, 6))
    plt.plot(history['loss'])
    plt.title('Training Loss')
    plt.savefig('loss_curve.png')

    # Logger le graphique
    mlfow.log_artifact('loss_curve.png')

    # Ou logger directement la figure
    mlfow.log_figure(plt.gcf(), 'loss_curve.png')
```

Autolog

MLflow peut logger automatiquement pour certains frameworks.

- **Scikit-learn autolog :**

```
import mlflow

# Activer autolog
mlflow.sklearn.autolog()

# Entraîner normalement - tout est loggé automatiquement !
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
# Paramètres, métriques et modèle loggés automatiquement
```

Frameworks supportés

```
# TensorFlow/Keras
```

```
mlflow.tensorflow.autolog()
```

```
# PyTorch
```

```
mlflow.pytorch.autolog()
```

```
# XGBoost
```

```
mlflow.xgboost.autolog()
```

```
# LightGBM
```

```
mlflow.lightgbm.autolog()
```

```
# Spark
```

```
mlflow.spark.autolog()
```

Tags et métadonnées

- Ajouter des tags :

```
with mlflow.start_run():  
    mlflow.set_tag("model_type", "classification")  
    mlflow.set_tag("developer", "john.doe")  
    mlflow.set_tag("version", "v1.0")  
    mlflow.set_tag("production_ready", "true")
```

Tags et métadonnées

- Tags multiples :

```
tags = {  
    "team": "data-science",  
    "project": "customer-churn",  
    "priority": "high"  
}  
  
with mlflow.start_run():  
    mlflow.set_tags(tags)
```

Bonnes pratiques

- Organisation des experiments :

```
# Bien - noms descriptifs
mlflow.set_experiment("customer-churn-classification")
mlflow.set_experiment("recommendation-system-v2")

# Éviter - noms vagues
mlflow.set_experiment("test")
mlflow.set_experiment("experiment1")
```

Bonnes pratiques

- **Nommage des runs :**

```
# Avec run_name
with mlflow.start_run(run_name="rf-tuned-hyperparams") :
    pass

# Sans nom (génère un UUID)
with mlflow.start_run():
    pass
```

Bonnes pratiques

- **Logger systématiquement**

```
with mlflow.start_run():  
    # Logger TOUT ce qui est important  
    mlflow.log_params(params)  
    mlflow.log_metrics(metrics)  
    mlflow.log_artifact("config.yaml")  
    mlflow.sklearn.log_model(model, "model")  
    mlflow.set_tags(tags)
```

